

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	Assessment:	Assessment Tool 2 – Flowchart-To-Code Conversion
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	A.Sathwika Reddy	Reg. No.:	192525274
Section / Batch:	2025	Date:	02/09/2026

Code of Conduct

I A.Sathwika Reddy-192525274 (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A.Sathwika Reddy

Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

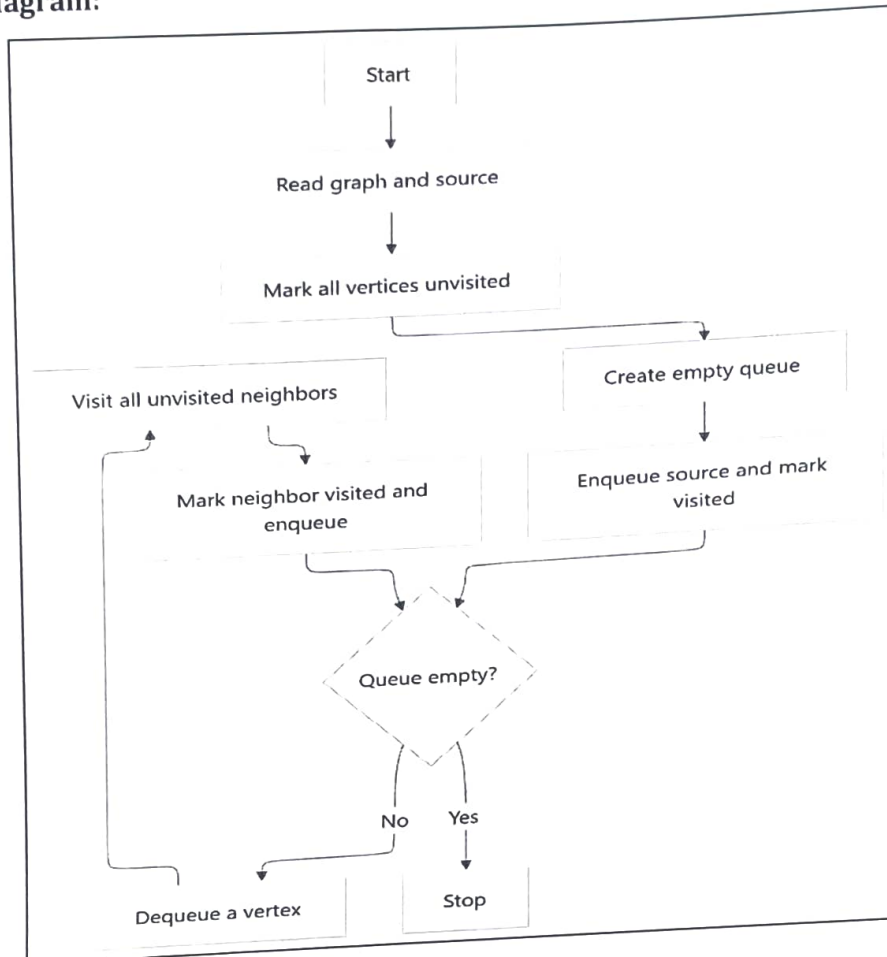
Section / Topic		Focus	Q Nos	Marks
Graph Traversal		BFS	1	20
Minimum Spanning Tree		Kruskal's Algorithm	2	20
Graph Sorting		Topological Sort	3	20
Graph Traversal		DFS	4	20
Minimum Spanning Tree		Prim's Algorithm	5	20
GRAND TOTAL:			100 Marks	

Annexure A – Flowchart-To-Code Conversion

Convert the flowchart for Breadth First Search traversal into code.

(20 Marks)

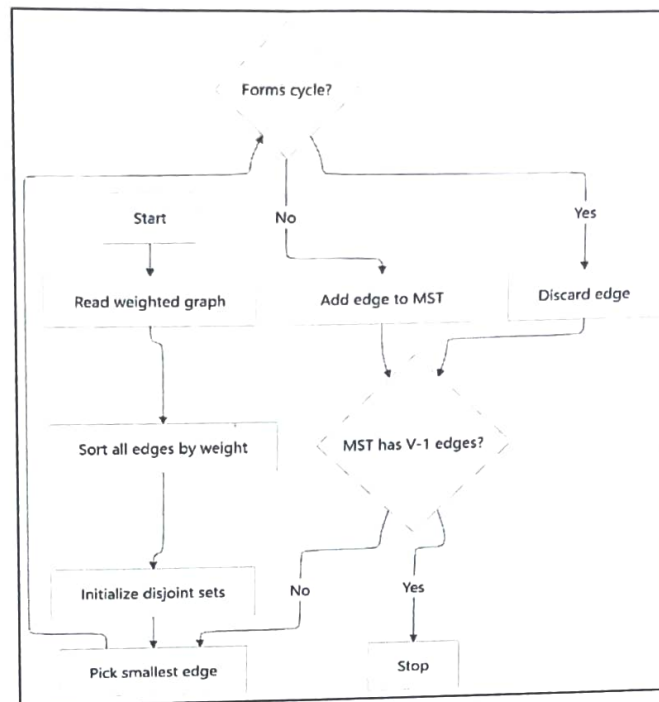
Flowchart Diagram:



2. Convert the flowchart for Kruskal's algorithm into code.

(20 Marks)

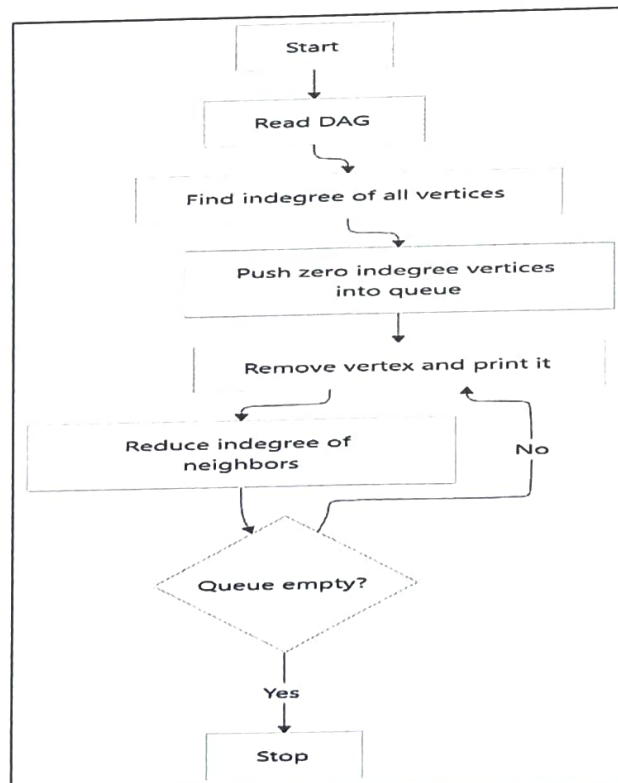
Flowchart Diagram:



3. Convert the flowchart for topological sort into code.

(20 Marks)

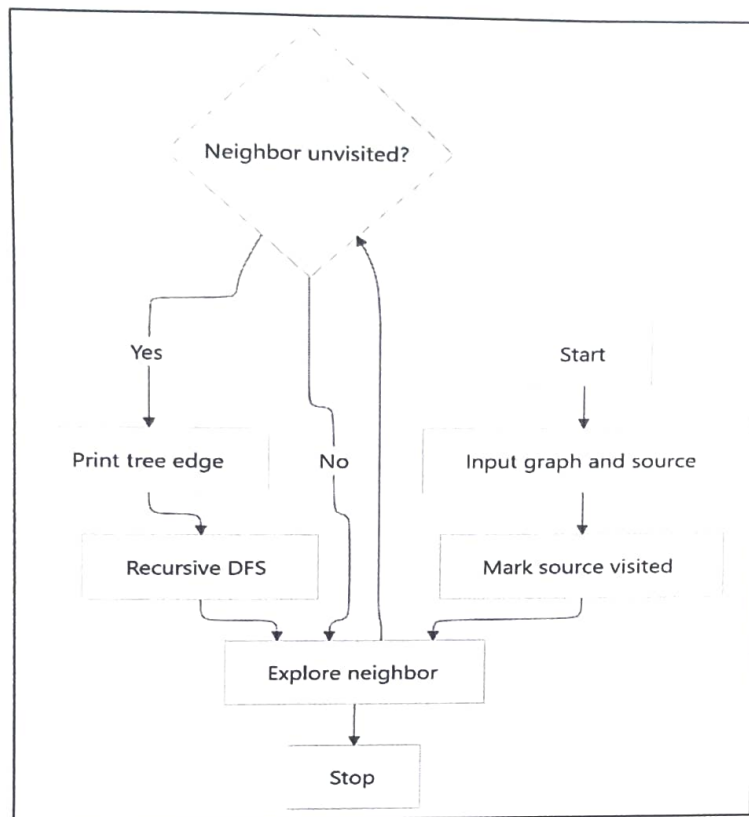
Flowchart Diagram:



4. Convert the flowchart for generating a DFS tree into code.

(20 Marks)

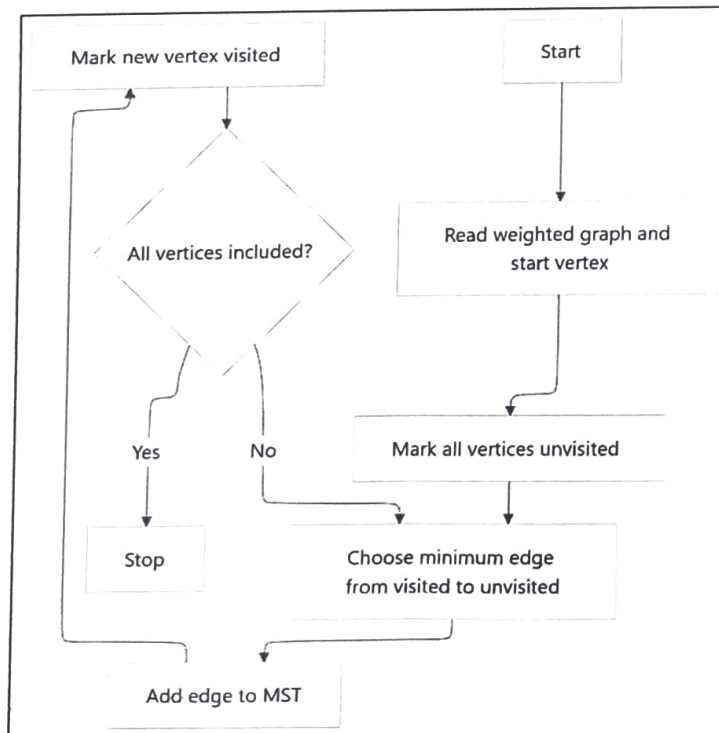
Flowchart Diagram:



5. Convert the flowchart for Prim's algorithm into code.

(20 Marks)

Flowchart Diagram:



Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1) BFS - Breadth - First Search

```
#include <stdio.h>

int main() {
    int n, a[10]
    [10], v[10] = {0}, q[10], f=0, r=0, s, i, j;
    scanf ("%d", &n);
    for (i=0; i<n; i++)
        for (j=0; j<n; j++)
            scanf ("%d", &a[i][j]);
    scanf ("%d", &s);
    v[s] = 1; q[r++] = s;
    while (f < r) {
        s = q[f++];
        printf ("%d", s);
        for (i=0; i<n; i++)
            if (a[s][i] && !v[i])
                v[i] = 1, q[r++] = i;
    }
}
```


2) Kruskal's Algorithm

```
#include <stdio.h>
```

```
int P[20], find(int x){return P[x]==x?
```

```
x : (P[x] = find(P[x])) ; }
```

```
int main() {
```

```
int n, m, i, j, u, v, w, c=0, t, eu[50], ev[50],  
ew[50];
```

```
scanf ("%d%d", &n, &m);
```

```
for (i=0; i<m; i++)
```

```
scanf ("%d%d%d", &eu[i], &ev[i], &ew[i]);
```

```
for (i=0; i<n; i++)
```

```
P[i] = i;
```

```
for (i=0; i<m; i++)
```

```
for (j=i+1; j<m; j++)
```

```
if (ew[i] > ew[j]) {
```

```
t = ew[i]; ew[i] = ew[j]; ew[j] = t;
```

```
t = eu[i]; eu[i] = eu[j]; eu[j] = t;
```

```
t = ev[i]; ev[i] = ev[j]; ev[j] = t;
```

```
}
```

```
for (i=0; i<m && c<n-1; i++) {
```

```
u = find (eu[i]); v = find (ev[i]);
```

```
if (u != v) {
```

```
printf ("%d - %d\n", eu[i], ev[i]);
```

```
P[u] = v; c++;
```

```
}
```

```
}
```

```
}
```

3) Topological Sort

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int n, a[10]
```

```
[10], in[10] = {0}, q[10], f=0, r=0, i, j, v;
```

```
scanf("%d", &n);
```

```
for(i=0; i<n; i++)
```

```
for(j=0; j<n; j++) {
```

```
scanf("%d", &a[i][j]);
```

```
if(a[i][j] != 0) in[j]++;
```

```
}
```

```
for(i=0; i<n; i++)
```

```
if(!in[i]) q[r++] = i;
```

```
while(f < r) {
```

```
v = q[f++];
```

```
printf("%d", v);
```

```
for(j=0; j<n; j++)
```

```
if(a[v][j] != 0) --in[j];
```

```
q[r++] = j;
```

```
}
```


48

DFS Tree

#include <stdio.h>

int a[10][10], v[10], n;

void dfs (int x) {

int i;

v[x] = 1;

printf ("%d", x);

for (i=0; i<n; i++)

if (a[x][i] && !v[i]) {

printf (" (%d-%d)", x, i);

dfs(i);

}

}

int main() {

int i, j, s;

scanf ("%d", &n);

for (i=0; i<n; i++)

for (j=0; j<n; j++)

scanf ("%d", &a[i][j]);

scanf ("%d", &s);

dfs(s);

}

5> Prim's Algorithm

```
#include <stdio.h>
```

```
#define I 999
```

```
int main() {
```

```
    int n, a[10]
```

```
    [10], k[10], p[10], v[10] = {0}, i, j, m, u;
```

```
    scanf("%d", &n);
```

```
    for(i=0; i<n; i++)
```

```
        for(j=0; j<n; j++) {
```

```
            scanf("%d", &a[i][j]);
```

```
            if(!a[i][j] && i != j)
```

```
                a[i][j] = I;
```

```
        }
```

```
    for(i=0; i<n; i++) k[i] = I;
```

```
    k[0] = 0;
```

```
    for(i=0; i<n; i++) {
```

```
        m = I;
```

```
        for(j=0; j<n; j++)
```

```
            if(!v[j] && k[j] < m)
```

```
                m = k[j], u = j;
```

```
        v[u] = 1;
```

```
        for(j=0; j<n; j++) if(!v[j] && a[u][j] < k[j])
```

```
            k[j] = a[u][j], p[j] = u;
```

```
    }
```

```
    for(i=1; i<n; i++)
```

```
        printf("%d %d\n", p[i], i);
```

```
}
```